

*Notes on Combinatorial and Probabilistic
Algorithms*
Riccardo Lo Iacono

July 8, 2026

Contents.

1	The analysis of Algorithms	1
1.1	Asymptotic notation	1
1.2	Amortized analysis	1

– 1 – The analysis of Algorithms.

As a good computer scientist knows, not all algorithms are the same nor are the resources these use; it is for these reasons that we need ways to analyze algorithms. In what follows, we first recall what asymptotic notation is, and then introduce the notion of *amortized analysis*.

– 1.1 – Asymptotic notation.

That idea behind asymptotic notation is that of analyzing algorithms, with respect to a given resource, more often than not being time, without caring about constants.

We recall the existence of three major notations: the *omega* notation (Ω), the *theta* notation (Θ) and the big-o notation ($\mathcal{O}$). Let $g(n)$ and $f(n)$ be two functions, then we define each notation as follows:

$$\Omega(f(n)) = \{g(n) \mid \exists c > 0, n_0 \geq 0 \text{ s.t. } g(n) \geq cf(n), \forall n \geq n_0\}$$

$$\Theta(f(n)) = \{g(n) \mid \exists c_1, c_2 > 0, n_0 \geq 0 \text{ s.t. } c_1f(n) \leq g(n) \leq c_2f(n), \forall n \geq n_0\}$$

$$\mathcal{O}(f(n)) = \{g(n) \mid \exists c, n_0 \geq 0 \text{ s.t. } g(n) \leq cf(n), \forall n \geq n_0\}$$

Although we give the definition in set notation, we write $g(n) = \mathcal{O}(f(n))$ to mean $g(n) \in \mathcal{O}(f(n))$; we do similarly for the other notations. Additionally, unless stated otherwise, assume we are interested in the time complexity of the algorithm.

– 1.2 – Amortized analysis.

Let us consider the following: assume we are given a data structure, and we are asked to compute a sequence of m operations on it; what is the overall complexity of all m operations, i.e., what is the time needed to compute all m operations.

It follows easily that, applying a worst case analysis, we need $m \cdot T_{MAX}(n)$, where $T_{MAX}(n)$ is the time complexity of the most expensive operation. For instance, under the worst case assumption, a sequence of m deletions from a tree has a complexity of $\mathcal{O}(m \cdot n)$.

We now introduce the notion of *amortized analysis*, and show how, over a sequence of operations, the real time complexity of an algorithm differs from the worst case one.

In its essence, amortized analysis does nothing more than averaging the complexity of all operations. That is, if $T_1(n)$ is the complexity of the first operation, $T_2(n)$ the one for the second operation, ..., $T_m(n)$ the complexity

Section 1 – The analysis of Algorithms

of the m -th operation, then the amortized time is given as

$$T_{\text{amortized}}(n) = \frac{1}{n} \sum_{i=1}^m T_i(n).$$

In other words, amortized time represents the time needed for each of the m operations.

The next section describes a simple ways to perform amortized analysis.

– 1.2.1 – Accounting method.

The core idea of the method follows that used by banks. In more details: to each operation we assign a cost, which represents its *amortized cost*. When the amortized cost of an operation exceeds its real cost, we are left with some credit that we can use to help pay other operations.

Example: Consider we need an algorithm to increment a n bits counter by an amount m . From a worst case analysis perspective, the algorithm needs a quadratic time, as the n -th bit may need to be flipped m times. It comes with no surprise that such a cost is well above the real one, as we are about to see.

Let us proceed this way: for each bit flip, assume that we are asked to pay a coin; then we proceed by paying two coins when flipping a bit to 1. The cost of the whole algorithm is easily seen to be given by the number of total bit flipped. But the following can be observed: the first increment, leaves us with a coin of credit, the second one does the same, etc. Thus, each operation requires a constant amount; that is, the amortized cost of the whole algorithm is $\mathcal{O}(n)$.

Remark: Other methods to perform amortized analysis exists, and their use varies with the algorithm we need to analyze. Though, all follow the same core idea: we overpay for some operations and use the credit thus produced to pay for others.